

Design and Verification of a Cache Coherency Verification IP for AMBA ACE Protocol Using UVM

1st Ketki Purushottam Kasaralika
VLSI and Embedded Systems
COEP Technological University
Pune, India
kasaralikarkp22.extc@coeptech.ac.in

2nd Vaishali Ingale
Department Of E & TC
COEP Technological University
Pune, India
vvi.extc@coeptech.ac.in

3rd Ashlesha Gokhale
ESCICOMP INDIA
PVT. LTD
Pune, India
ashleshaag@gmail.com

4th Vanita Agarwal
Department Of E & TC
COEP Technological University
Pune, India
vsa.extc@coeptech.ac.in

Abstract—The ever evolving architecture of SoCs has created the need for a robust communication protocol to maintain the data consistency and data integrity between multi-master SOC. This obstacle has been overcome by the AMBA ACE protocol. In the proposed Verification IP, the AMBA ACE protocol has been verified with the help of a write-through, two-way set associative cache which has the cache line size of 32 Bytes, supports both read and write allocate. The VIP has been verified in three different phases defined as read phase, write phase and snoop phase. The simulation of the implementation was completed in Cadence Xcelium Logic Simulator tool.

Index Terms—AMBA AXI, AMBA ACE, Coherency Protocol, UVM, SystemVerilog, Verification IP.

I. INTRODUCTION

The need for a versatile, adaptable, and coherent communication protocol is growing due to the increasing level of sophistication and heterogeneity of modern SoCs. In response, the Advanced Microcontroller Bus Architecture (AMBA) AXI Coherency Extension (ACE) protocol is a key advancement in computer architecture and system-on-chip (SoC) design [1]. The AMBA ACE protocol provides improved support for coherency, flexibility, and configurability, which makes it highly suitable for a wide range of system architectures and applications. As an extension of the widely used AXI protocol, AMBA ACE provides additional snooping channels in addition to channels that are already available. This specialization increases throughput and power efficiency, lowers latency, and improves overall coherency in complex SoC settings. The AMBA ACE protocol relies strongly on the coherence mechanism, which is similar to the 5-state coherence protocol MESI, to optimize transactions in the network and maintain coherency in a multi-master architecture.

In this paper, we primarily verify the protocol using the Universal Verification Methodology (UVM). This paper uses the verification IP of a cache slave device to improve the visualization of the data's coherence and integrity. An external snooping master and an internal AXI master are used in the implementation to assist the model and evaluate the coherence of a multi-master SOC.

II. RELATED WORKS

An extension of the AXI protocol based on snooping is called the AXI Coherency Extensions (ACE) network. It uses a cache state architecture to provide improved support for cache coherency and higher configurability. There are five cache states in total: invalid, unique clean, unique dirty, shared clean, and shared dirty. The invalid state depicts that a cache line for a given address is not saved in cache. The shared states indicate that there are several copies of this cache line in the shareable domain, whereas unique state shows that there is only one copy. While clean displays consistent with main memory, dirty states identify the cache lines that have been altered in relation to main memory [2]. This protocol offers a reliable fix for a range of system configurations and applications.

The ACE protocol, like AXI supports different channels for various operations. It has three channels dedicated for its write transaction as follows: control signals on AW channel, data on W channel and the response of the transaction on B channel. Similarly it has two channels dedicated for read transaction AR and R but unlike its write counterpart it sends response on the data channel itself. The addition for ACE as compared to AXI are the three snooping channels similar to that of write channel. They are control channel for snoop, data and

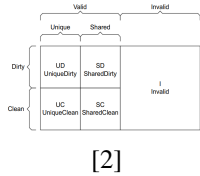


Fig. 1: ACE cache state model



Fig. 2: ACE Channels

response defined as AC channel, CD channel and CR channel respectively. While AC channel is started by an external master, channels AR and AW are started by the cache verification IP's internal master. All of the channels in the protocol follow the handshaking dependency that is VALID and READY of the channel needs to be high at the same time for the transfer to happen [2].

The information about the transfer's course is transferred through the Read and Write control channels. AxSIZE, AxLEN, AxBURST, AxCACHE, AxID, AxADDR, and snooping-specific signals like ACBAR, ACDOMAIN, and ACSNOOP are some of these signals. The signal AxLEN indicates how many beats(transfers) will be required for the transaction, whereas the signal AxSIZE indicates how many bytes will be transferred in a single beat [2]. The address of the location where the transaction will take place is transferred via the AxADDR bus. The type of data transfer that will take place —fixed, wrap, or increment—is determined by the signal AxBURST [2]. Transfers to a cache memory are handled through burst type wrap. The AxCACHE bus, which consists of four bits and defines buffer-able, cache-able, and its allocation policy, defines the type of memory to which the interaction will take place [2]. We have chosen write-through cache that supports both read and write allocation in this cache VIP implementation. The types of snoop transactions that can happen are defined by the AxSNOOP bus for the ACE extension, and they include Read Clean, Read Shared, Clean Unique, Make Unique, Write Unique Full, Write Back, Evict, and more. Transferring data is not a must for every snoop transaction [2]. The snoop control channel doesn't include the signals for size, length, burst type and cache type.

Signals like ID, data, strobe are used in the data channel of AXI for write phase, alongwith and acknowledgement for ACE write data channel. ID, data, last, and response signals are supported on the read data channel [3]. The Snoop data channel supports just a few signals, such as data and last. The data available on the address specified by the control channel is transferred using the data bus in each of these channels. In this transaction, the last signal indicates which data on the data bus is the final beat. Whether a transaction was successful or unsuccessful is indicated by the response bus. A signal acknowledgement indicates the completion of an ACE transaction. ID and response signals are part of the write response channel. On the other hand, the snoop response channel lacks an ID signal. The operation of the response bus in these two channels is identical to that of the response bus

in the read data channel [2].

The ACE protocol tries to serves various types of cache memories by supporting different types of snooping variations. It can facilitate Write Through and Write Back cache and it can change snooping transaction to accommodate MESI or MOESI [2]. In the implementation for this paper we have used the snooping transactions which satisfy a write through cache in the read and write phases of the verification IP. All the snoop transactions that are received on snoop channel are serviced by the cache verification IP.

The implementation in the paper [3] verifies the AXI protocol for all its burst types with varying burst length and size whereas the implementaion in paper [8] verifies the AHB-Lite to AXI Bridge for the increment burst type of varying burst length. The paper [5] had used a write back cache to verify the ACE protocol but had not verified the cache state transitions with each transactions. The model in paper [9] uses the ACE protocol in the prefetching mechanism to help improve Last Level Cache (LLC) utilization in PCIe based I/o subsystems. The proposed implementation in this paper verifies the ACE Protocol with the help of a practical model of a 32B write-through cache with read and write allocate, and LFU re-allocation policy. It also aims to show cache line state transitions with each transactions.

III. METHODOLOGY

A. Need for UVM

The increasing level of sophistication and heterogeneity of modern SoCs along with the continuous search for faster, power and area efficient designs which are also accurate has raised the likelihood of failure. As a result, the chip designers are resolute that it is essential to give both the design and verification processes top priority. For the functional verification of models written with hardware descriptive languages, SystemVerilog is commonly used. Nevertheless, it is not perfect in several important aspects, including as scalability, reusability, and interface with other software tools. Because there are no established programming standards, testbenches are created on an as-needed basis using pure SystemVerilog. It follows that these testbenches are not entirely reusable. It's also difficult to interpret and update the code [4].

In the industry, three rival approaches arose: Mentor's Open Verification Methodology (OVM), Cadence's Universal Reuse Methodology (URM), and Synopsys' Verification Methodology Manual (VMM). All of them claimed to enable re utilization code and provided a base class library along with TLM level modeling capabilities causing mass confusion [4].

However, Accelera selected the Open Verification Methodology (OVM) over others, leading to the birth of the Universal Verification Methodology (UVM) [4]. This solution was chosen because of its standardized methodology which had enhanced interoperability, better modularity, and better reporting and debugging features. This methodology makes it better than SystemVerilog. Most language features were carried over from OVM to UVM. The base class library provides all of the essential classes like driver, monitor, etc.,

along with predefined methods called `uvm_phases` which help streamline the testbench. Both of these facilitate development of reliable and scalable test environments. The methodology encourages engineers to construct broad, reusable Verification IPs and Testbenches. This also helps with easy maintenance of the testbench. UVM also supports an enhanced reporting mechanism whose verbosity can be changed as per requirement. This feature enriches the traceability and visibility of the whole process, facilitating effective debugging and analysis. The paper [8] also highlights this difference by using the same DUT but using both SystemVerilog and UVM and proves that UVM aids in reducing time-to-market by reducing verification time and effort.

B. Architecture

After learning about the characteristics of UVM, let's take a closer look at the VIP's architecture that was created specifically for this application. The VIP as discussed earlier consists of an internal AXI master and slave, cache memory and ACE driver and monitor. This implementation also uses an external snooping master to verify the functionality of AC channel accurately.

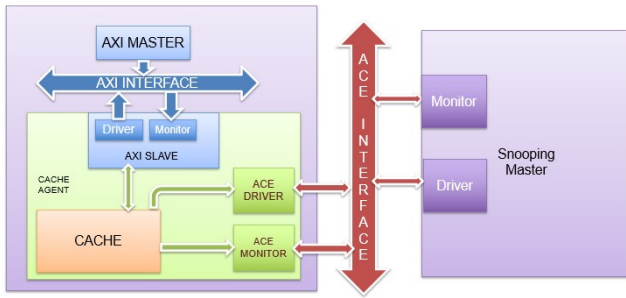


Fig. 3: Architecture of implemented Verification IP

1) *AXI Master*: To access the cache and create transactions for it, the AXI master is constructed of the following parts. The component AXI transaction declares all the signals that are to be sent to AXI interface alongside the constraints necessary for the test scenarios described in verification plan. Sequence component carries out the randomization of signals as per defined constraints. Two different sequences for read and write phases are defined for this implementation. Similarly, the components Sequencer sends data to the driver when needed and is also responsible for synchronization with the slave and Driver drives the randomized data generated by sequence component to the AXI interface. Agent class defines the integration of sequencer and driver. The connection of their port and export is also done in connect phase [3].

2) *AXI Slave*: To service the read and write requests from AXI master, two components are used. Monitor is used to receive the control information for both phases and the data during write phase. A driver to send data for read transactions. This component is a part of cache agent class which is the integral part of this VIP.

a	axi_agent	-	@2038
drv	axi_driver	-	@2133
rsp_port	uvm_analysis_port	-	@2238
seq_item_port	uvm_seq_item_pull_port	-	@2186
seqr	axi_sequencer	-	@2271
rsp_export	uvm_analysis_export	-	@2331
seq_item_export	uvm_seq_item_pull_imp	-	@2891
arbitration_queue	array	0	-
lock_queue	array	0	-
num_last_reqs	integral	32	'd1
num_last_rsp	integral	32	'd1

Fig. 4: Elaboration of Agent class

3) *Cache Memory*: In order to accurately visualize the working of the protocol, the implementation uses a modelled cache memory. The memory can be accessed through both interfaces, namely AXI and ACE, by their respective drivers and monitors. The VIP cache model has the following specifications :

- Cache Line Size : 32B
- Cache Size : 2KB
- Cache Mapping : Two-way Set Associative
- Write Policy : Write-Through
- Allocation Policy : Read and Write allocate
- Relocation Policy : Least Frequently Used (LFU)

Only three of the five cache states specified by the protocol —invalid, unique clean, and shared clean—will be supported by the write-through cache because it will always be cohesive with main memory. This is also a part of cache agent class.

cag	cache_agent	-	@2071
cache	trial_cache2	-	@3128
recvd_link_axi	uvm_blocking_transport_imp	-	@3208
recvd_mon_axi	uvm_blocking_get_port	-	@3260
recvd_write_axi	uvm_blocking_put_imp	-	@3312
send_driver_ace	uvm_blocking_put_port	-	@3362
send_driver_ace_read	uvm_blocking_put_port	-	@3412
cdrv	cache_driver	-	@3443
recvd_ace_cache	uvm_blocking_put_imp	-	@3639
recvd_ace_cache_read	uvm_blocking_put_imp	-	@3689
rsp_port	uvm_analysis_port	-	@3541
seq_item_port	uvm_seq_item_pull_port	-	@3492
cmom	cache_monitor	-	@3572
send_cache	uvm_blocking_get_imp	-	@3576
sd	axi_driver	-	@2993
rsp_port	uvm_analysis_port	-	@3095
send_read	uvm_blocking_put_imp	-	@3819
seq_item_port	uvm_seq_item_pull_port	-	@3043
axi_s_mon	axi_s_mon	-	@3257
send_driver_axi	uvm_blocking_put_port	-	@3995
send_link	uvm_blocking_transport_port	-	@3875
send_write	uvm_blocking_put_port	-	@3925

Fig. 5: Elaboration of cache agent class

4) *ACE driver*: A cache line in shared state must be updated in main memory and invalidated in the system's snooping masters in order for the system to function coherently. [2]. This is done by the ACE driver inside the VIP which is a part of agent class of cache. The snoop response and snoop data channel are also driven by this driver.

5) *ACE monitor*: The snoop transactions are received in this component and sent to the cache for further processing. If there is a cache miss during the read phase of the AXI master, the snooping masters are asked to share the cache line on the ACE interface, if it exists [2]. This component is responsible for generating such a request and then receiving the data sent by the snooping master. This is also a part of cache agent class.

```

graph TD
    START([START]) --> GenTrans[Generation of Transaction at AXI master]
    GenTrans --> RecInfo[VIP AXI Slave receives information]
    RecInfo --> Read{Read?}
    Read -- Write --> SendStore[Send to Cache for store]
    SendStore --> Hit1{Hit?}
    Hit1 -- Miss --> EmptyCache{Empty cache available?}
    EmptyCache -- Full --> Evict[Evict Least Frequently used cache line]
    Evict --> Invalidate[Invalidate other caches and update main memory by sending WriteUniqueFull]
    EmptyCache -- Empty --> StoreEmpty[Store in empty space]
    StoreEmpty --> Invalidate
    Hit1 -- Hit --> StoreLine[Store cache line]
    StoreLine --> Invalidate
    Invalidate --> SendData[Send Data on AXI interface to master]
    SendData --> END([END])
    SendData --> RecCache[Receive cache line from external master]
    RecCache --> Hit2{Hit?}
    Hit2 -- Miss --> SendInfo[Send Information on ACE interface]
    SendInfo --> SendData
    Hit2 -- Hit --> SendData
  
```

```

graph TD
    Start([START]) --> Gen[Generation of Snoop Transaction at External master]
    Gen --> Rec[VIP ACE Monitor receives snoop information]
    Rec --> AC{AC Read ?}
    AC --> Read[Read Cache and change its state]
    AC --> Change[Change cache state]
    Read --> SendCR[Send Response on CR channel]
    SendCR --> SendCD[Send Data if available on CD channel]
    SendCD --> End([END])
    Change --> SendCR2[Send response on CR channel]
    SendCR2 --> End
  
```

6) *External Snooping master:* The agent of this component services all the ACE transactions generated by the VIP. This also has a sequence item component to generate the snoop transactions for the VIP and a transaction class to define them. The driver in this component is responsible for sending data and in read phase of VIP, the response for write phase of the VIP and for driving the snoop address channel when a snoop transaction at it's own end is generated

The efficient and precise verification of any protocol needs to make sure that all the fundamental features of that protocol, including handshaking and transaction attributes, are verified. The verification plan of this VIP is divided into three phases : Write, Read and Snoop phase, which are further described in following sections.

1) *Phase I: Write Phase:* The write phase has the write addresses, in AXI master defined, in a way that are randomized only within a certain set of aligned addresses. This constraint is added to create an overlap between all the phases to generate maximum cache hits. The flowchart in Fig.7 defines how the a write transaction generated inside AXI master is serviced by the cache. When a write transaction is generated by AXI master, the AXI slave first receives the control information, then saves the transmitted data and gives appropriate response. The snooping transaction Write Unique Full works well in a write-through cache since it is used to update the main memory and invalidate the snooping masters [2]. Therefore, it is then used by the cache on the ACE interface.

2) *Phase II: Read Phase:* The read phase has constraint randomization of addresses which test two scenarios, cache hit and cache miss. When a cache hit occurs, the cache is read and sent to AXI master by the driver component in the AXI slave. Whereas for a cache miss, the VIP generates a ACE

transaction to request the cache line from snooping masters. The cache modelled in this VIP being a write-through cache it does not support the responsibility of writing back a dirty cache. Hence, the read transaction to snoop other masters has to be Read Clean or Read Shared Not Dirty.

3) *Phase III: Snoop Phase:* This phase uses the snoop-master to generate transaction. This has randomized snoop type signal which may generate any of the following transaction- Read Clean, Read Unique, Read Shared, Read Shared Not Dirty, Read Once, Clean Shared, Clean Invalid and Make Invalid. When these transactions are detected at the ACE monitor of the VIP they are serviced in one of the two ways as shown in the Fig.8.

IV. RESULTS

The results are illustrated as per the defined phases in the verification plan III-C. The simulation of this VIP is done on Cadence Xcellium Logic Simulator tool. Although the AXI interface's memory element for all burst kinds has been confirmed by earlier work [3] cited in this paper, we only employ wrap burst in this implementation because it is compatible with cache memory. This can be verified in the Fig. 9 AXI interface with axburst as the value 2. This implementation has also generated only aligned randomized addresses which are required for wrap type bursts. The size and length have also been randomized in the paper [3] whereas we have kept it constant to be compatible with the cache line size of 32 Bytes. The previous work [5] had used the cache states to be independent and not showing transitions with snoop data or address but we have shown the changing cache states of the VIP in each of the phase in Fig.11, Fig. 15 and 16 depicted by the signal `vip_cache_state`

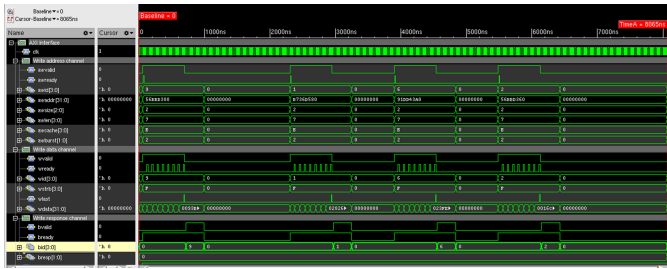


Fig. 9: AXI Interface for Write phase

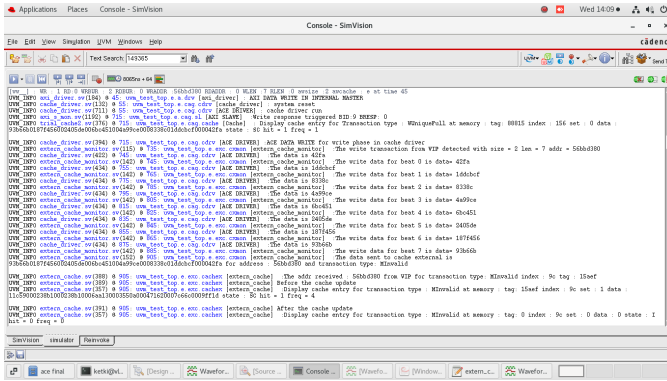


Fig. 10: Write transaction on console

A. Phase I: Write Phase

1) **AXI Interface:** The simulation window of AXI interface in Fig.9 records the write transaction between AXI master and AXI slave. As observed in Fig.9, The control signals are first sent on AW channel, which are received by the AXI slave only when the AWVALID and AWREADY signals are high [2]. This can also be verified by the console window in Fig.10. The data is then transferred to slave in eight beats of 4 Bytes each as specified by the control channel, which can also be seen in the Fig.9. The response from slave is sent on the B channel and it will be OKAY as the transaction was successful. The cache line changes when the response is generated and stored in cache, but before ACE transaction, as observed in Fig.11,10.

2) **ACE Interface:** The simulation window in Fig.11 documents the ACE interface of the write phase and it also verifies that handshaking conditions are adhered. Given that the cache line is of 32 Bytes the ACE transaction has the signals size and length to be two and seven respectively. The ACE transaction Write Unique Full will always change the cache state to Shared Clean [2].

B. Phase II: Read Phase

1) **AXI Interface:** The simulation window in Fig.12 shows two addresses, one for a cache miss and one for cache hit. When a cache hit takes place as documented in Fig.13, the data is sent immediately to the master. On the other hand, if a cache miss takes place as documented in Fig.14, the slave has to wait for the ACE transaction to complete before sending data on AXI interface.

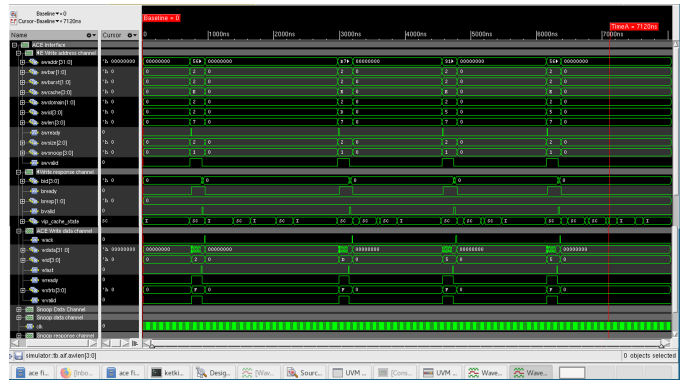


Fig. 11: ACE Interface for Write Phase

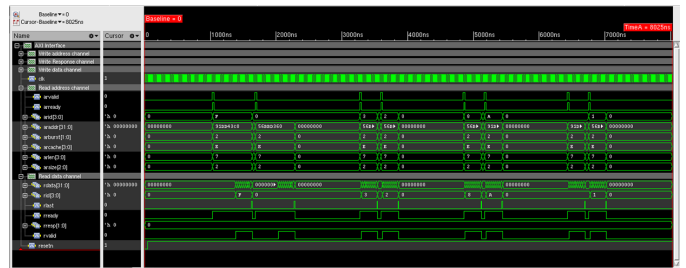


Fig. 12: AXI Interface for Read phase

2) **ACE Interface:** The ACE transaction is generated only in event of a cache miss. This transaction can be verified with the simulation window in Fig.15. Due to the Read Clean transaction used the 3rd bit of read response, which indicates if a cache line is passed dirty, should be zero. The cache state changes after the ACE transaction receives a RACK.

C. Phase III: Snoop Phase

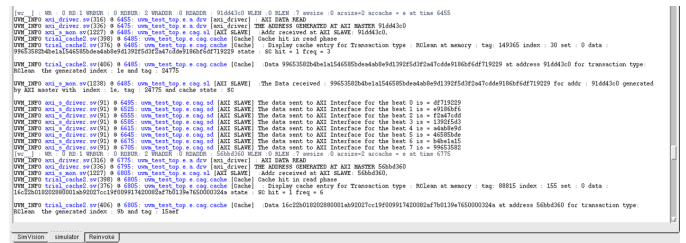


Fig. 13: Console For Cache Hit

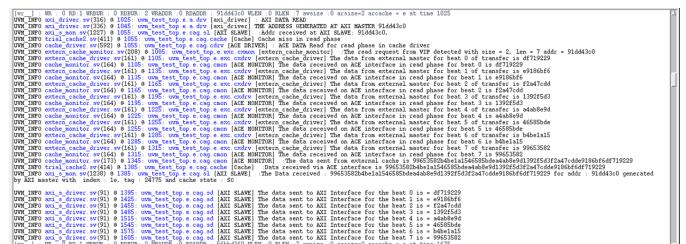


Fig. 14: Console for Cache Miss

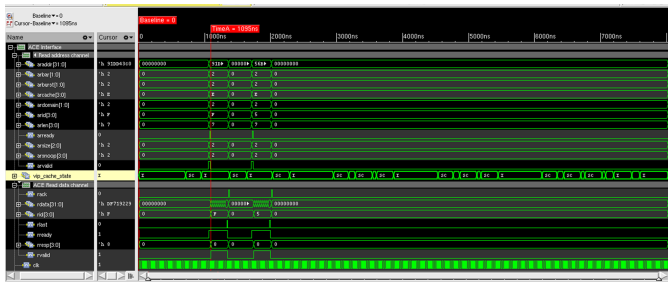


Fig. 15: ACE Interface for Read Phase

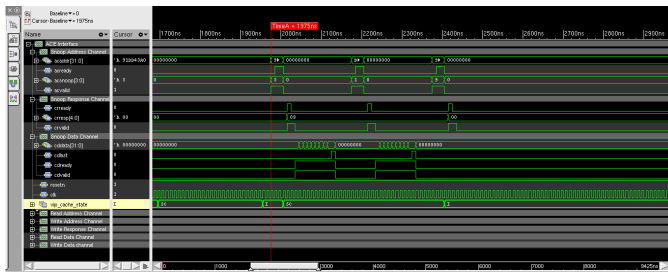


Fig. 16: ACE interface for Snoop phase

1) *ACE Interface:* The snoop transaction starts on the ACE interface is initiated by a snooping master and can be observed in the Fig.17. The snoop channel dependencies defined by the protocol are conformed to in this implementation and can be verified in the Fig.16. We can also observe in Fig.17 and Fig.16 that a for snoop read transaction the data is sent on the data channel and for a invalid transaction only a response is generated without a data transfer as defined by the protocol [2].

V. CONCLUSION

The Verification IP described in the architecture has been implemented as documented in the Fig.4, Fig.5 and Fig.6 on the console window. The phases defined in the verification plan have been accurately verified which are observed in the simulation results and console windows.

The state transitions of the cache line of VIP when handling any request can also be observed in the simulation waveform on the signal `vip_cache_state`.

It's important to note that the VIP has yet to explore other features of the protocol which would be used in a write-back cache and incorporating them would help the VIP implementation to be more robust.



Fig. 17: Console during Snoop Phase

VI. FUTURE WORK

The VIP has developed a write-through cache which can be further extended to write-back cache and can also be extended to have parameterized specifications. With even more complicated SoCs, AMBA CHI protocol is getting recognized as a better coherency protocol. This implementation could be further integrated with it by designing a bridge component between ACE and CHI.

REFERENCES

- [1] A. Kriouile, W. Serwe " Formal Analysis of the ACE Specification for Cache Coherent Systems-on-Chip " 18th International Workshop on Formal Methods for Industrial Critical Systems (FMICS), Madrid, Spain, 2013.
- [2] "AMBA® AXI and ACE Protocol Specification" Issue D, 2019.
- [3] A. S. Chandgaonkar, V. V. Ingale, V. Patil and V. Agarwal, "Development Of SV UVM Testbench For Verification Of AMBA AXI3 IP Used For Memory Access Application," 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), Delhi, India, 2023.
- [4] A. B. Mehta, "Chapter 4 UVM (Universal Verification Methodology)", ASIC/SoC Functional Design Verification, 2018
- [5] P. Dukare, A. Gokhale, V. Ingale "Development of AMBA ACE Protocol" 7th International Conference on Computing in Engineering & Technology (ICCET 2022), Online Conference, 2022.
- [6] A. B. Gamboa ,C. Chipres, S. Jadhav "UVM-Based Design and Verification of AHB-Lite to AXI Bridge" IEEE SoutheastCon 2024, Atlanta, Georgia, 2024.
- [7] R. Chandaluri, U. Nelakuditi "Design and Implementation of AXI4-lite Interface in Zynq SoC" 2022 IEEE Delhi Section Conference (DELCON), New Delhi, India, 2022.
- [8] A. R. Babu, P. Anand, Y. Kim, S. Jadhav "System Verilog versus UVM-based Verification of AXI4-Lite Arbitration" IEEE SoutheastCon 2023, Orlando, FL, USA, 2023.
- [9] W. Wang, P. Zhang "Prefetch-directed Scheme for Accelerating Memory Accesses of PCIe-based I/O Subsystem" 2023 5th International Academic Exchange Conference on Science and Technology Innovation (IAECST), Guangzhou, China, 2023.
- [10] S. Roozkhosh, D. Hoornaert, R. Mancuso "CAESAR: Coherence-Aided Elective and Seamless Alternative Routing via on-chip FPGA" 2022 IEEE Real-Time Systems Symposium (RTSS), Houston, TX, USA, 2022.
- [11] O. Pataskar, K. H. Gurjar, V. Ingale, A. Gokhale "AMBA AXI4 Interconnect Universal Verification Component" Turkish Online Journal of Qualitative Inquiry (TOJQI) Volume 12, Issue 9, 2021.
- [12] A. Hussien, S. Mohamed, M. Soliman, H. Mostafa "Development of a Generic and a Reconfigurable UVM-Based Verification Environment for SoC Buses" 31st International Conference on Microelectronics (ICM), Cairo, Egypt, 2019.
- [13] S. Sasikala, d P. Sivaranjani, T. Meeradevi, M. Keerthana "Design and Comparative Analysis of Open Core Protocol and Advanced Extensible Interface Protocol" Proceedings of the 8th International Conference on Communication and Electronics Systems (ICES 2023), 2023.
- [14] S. Malhotra, N. R. Prakash "UVM-Based Verification IP of AMBA AXI Protocol Showing Multiple Transactions and Data Interconnect" Proceedings of ICCDN Advances in Communication, Devices and Networking, 2018.
- [15] Accellera Systems Initiative, Universal Verification Methodology (UVM) 1.2 User's Guide, 2015.